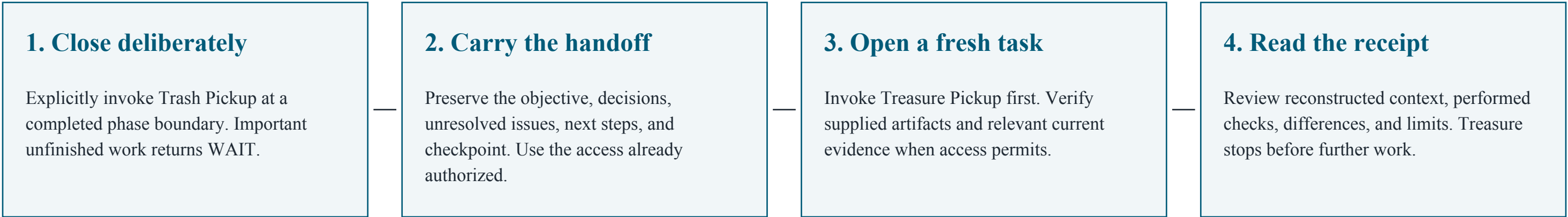


Trash Pickup + Treasure Pickup

Standalone edition: carry the decisions that matter, without installing DobeWorks/DEGS.



Figure 1
A deliberate context boundary with visible evidence



Normal authorized access

The saved workflow preserves package, checkpoint, registry and receipt behavior. Hash and identity checks compare recorded artifacts with the relevant current state. A conflict or failed check keeps its failure result.

Restricted access

The same content can travel through the conversation. Unavailable checks remain UNKNOWN or NOT RUN. Reconstruction can finish with explicit limits; it does not establish execution readiness or original-byte integrity.

Meaning of “trash.” Stale context, not project files. Neither skill deletes project data or grants authority to begin the next phase.

A receipt worth reading

Explicit invocation. Honest verification limits. No automatic continuation.



What the handoff carries

A bounded account of the work: objective, current phase, known-good state, decisions, relevant resources, warnings, next phase, next actions, and human actions required.

What the receipt makes visible

The reported outcome, observed drift, performed checks, unavailable evidence, and limits. An actual integrity failure cannot become success by switching to a restricted-access fallback.

What a hash establishes

Agreement with recorded bytes. It does not establish independent authorship, truth, or continuing validity after unobserved change. Artifact instructions cannot override the current user or host safeguards.

What readers can install

The staged repositories provide a paired installer and MIT licensing, with separate third-party tool notices. The installer was exercised on Apple silicon; access is still required while the repositories are private. This is a development snapshot, not a versioned public release.

Source basis: Trash main [bb44f27](#) and Treasure main [cb82e32](#); read their current README, SKILL, setup, security, and lifecycle documents for the operational contract. The figure explains that contract; it does not invoke either skill. Separating transfer from execution follows modular design; inspectable verification follows secure-development practice (Parnas, 1972; Souppaya et al., 2022).

References

Parnas, D. L. (1972). On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12), 1053-1058. <https://doi.org/10.1145/361598.361623>

Souppaya, M., Scarfone, K., & Dodson, D. (2022). *Secure software development framework (SSDF) version 1.1: Recommendations for mitigating the risk of software vulnerabilities* (NIST Special Publication 800-218). <https://doi.org/10.6028/NIST.SP.800-218>